فصل ۲۰: الگوهای معماری

در فصل ۹ میان سبک‌های معماری (architectural styles) و الگوهای معماری (architectural patterns) تمایز گذاشتیم. به‌طور خلاصه، **سبک‌ها** توپولوژی‌هایی نام‌دار هستند که معماران آن‌ها را بر پایهٔ تفاوت در توپولوژی، معماری فیزیکی، شیوهٔ استقرار، سبک‌های ارتباطی و توپولوژی‌های داده از هم متمایز می‌کنند. **الگوهای** معماری، که از کتاب پرارجاعِ **Design Patterns** الهام گرفته‌اند، راه‌حل‌هایی برای مسائل‌اند که در زمینه‌ای مشخص معنا می‌یابند.

مهم است میان الگوهای معماری و «بهترین شیوه‌ها» (best practices) تمایز بگذاریم (موضوعی که در فصل ۲۱ عمیق‌تر به آن می‌پردازیم). «بهترین شیوه» نامیدنِ چیزی به این معناست که معمار وظیفه‌ای روشن دارد: هر بار که وضعیت مشخصی پیش آمد، آن شیوه را به کار ببندد. اگر آن را شیوهٔ **بهتر** می‌نامیدیم، دست‌کم جایی برای بحث باقی می‌ماند؛ اما نه، آن را شیوهٔ **بهترین** می‌نامیم و به معماران اجازه می‌دهیم ذهنشان را خاموش کنند و همیشه از همان راه‌حل پیروی کنند.

همچنین مهم است میان الگوها و **راه‌حل‌ها** تمایز بگذاریم. بسیاری از ابزارها، چارچوب‌ها، کتابخانه‌ها و دیگر مصنوعات توسعهٔ نرم‌افزار یک یا چند الگو را در خود کپسوله می‌کنند؛ بسته به شیوهٔ پیاده‌سازی‌شان، با درجه‌های متفاوتی از وفاداری به الگو و درآمیختگی با الگوهای دیگر. نخست بر شناسایی مناسب‌ترین الگو تمرکز کنید و سپس مناسب‌ترین پیاده‌سازی را برای آن برگزینید.

در این فصل، نمونه‌هایی پراکنده از الگوهای شاخص را ارائه می‌کنیم تا در مقایسه با سبک‌هایی که در بخش دوم این کتاب معرفی کردیم، تمایز و زمینه را روشن کنند. نخستین نمونهٔ ما، یک الگوی بازاستفاده در معماری، تفاوت میان الگوها و پیاده‌سازی‌ها را روشن می‌کند.

بازاستفاده

تفکیک میان کوپلینگ حوزه‌ای (domain coupling) و کوپلینگ عملیاتی (operational coupling) یکی از دغدغه‌های رایج معماری در معماری‌های توزیع‌شده‌ای مانند میکروسرویس‌هاست.

جداسازی کوپلینگ حوزه‌ای و عملیاتی

یکی از اهداف طراحی در معماری‌های میکروسرویس، جداسازیِ حداکثری است که اغلب در این توصیه نمود می‌یابد: «دوباره‌نویسی بر کوپلینگ ترجیح دارد».

فرض کنید دو سرویس باید اطلاعات نمایه‌های مشتری را میان خود رد و بدل کنند، اما زمینهٔ محدودِ حوزه‌محورِ معماری اصرار دارد که جزئیات پیاده‌سازی باید برای هر سرویس خصوصی بماند. یک راه‌حل رایج این است که به هر سرویس بازنمایی درونیِ خودش از موجودیت‌هایی مانند Profile داده شود و سپس آن اطلاعات به شیوه‌هایی با

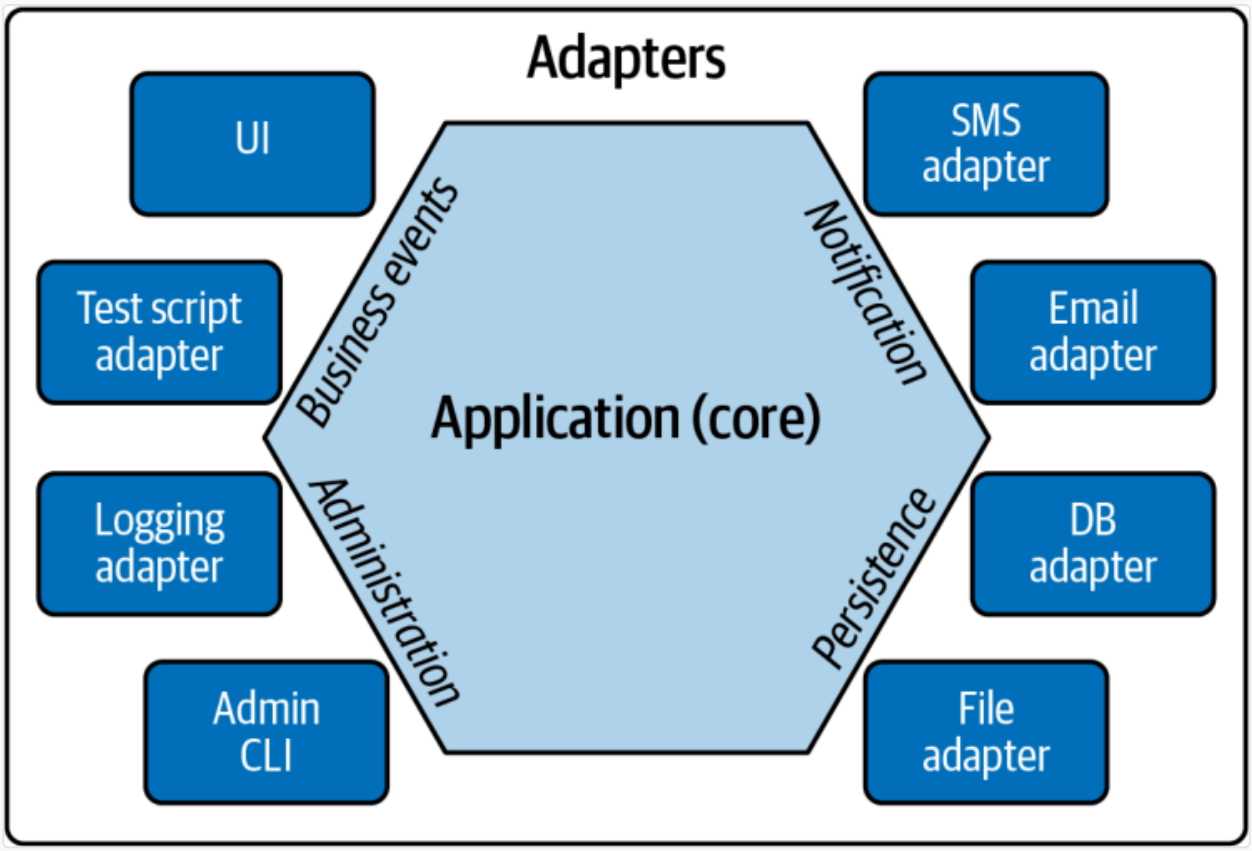
کوپلینگ سست `(loosely coupled)`، مانند جفت‌های نام-مقدار `(name-value pairs)` در `JSON`، منتقل شود. این کار به هر سرویس اجازه می‌دهد بازنمایی درونی خود، از جمله پشتهٔ فناوری، را هر طور که بخواهد تغییر دهد، بی‌آنکه یکپارچه‌سازی را بشکند. توسعه‌دهندگان معمولاً دوباره‌نویسیِ کد را ناپسند می‌دانند، زیرا می‌تواند مشکلاتی در هم‌گام‌سازی، رانش معنایی `(semantic drift)` و مانند آن پدید آورد؛ اما برخی چیزها از دوباره‌نویسی بدترند... و در میکروسرویس‌ها، کوپلینگ یکی از آن‌هاست.

معمارانی که میکروسرویس طراحی می‌کنند، معمولاً با این واقعیت کنار می‌آیند که گاهی برای حفظ جداسازی باید پیاده‌سازی‌ها را دوباره بنویسند. اما قابلیت‌هایی که از کوپلینگ بالا **سود می‌برند** چه؟ هر سرویس باید قابلیت‌های عملیاتیِ مشترکی مانند پایش، ثبت وقایع، احراز هویت و مجوزدهی، و قطع‌کننده‌های مدار داشته باشد. اما اجازه‌دادن به هر تیم برای مدیریت این وابستگی‌ها اغلب به هرج‌ومرج می‌انجامد.

برای مثال، شرکتی را در نظر بگیرید که می‌کوشد برای همهٔ سرویس‌هایش یک راه‌حل پایشِ استاندارد برگزیند تا عملیاتی‌کردنِ آن‌ها آسان‌تر شود. معمار تصمیم می‌گیرد هر تیم را مسئول پیاده‌سازیِ پایش برای سرویس خودش کند: تیم سرویس `Payment`، تیم سرویس `Inventory` و مانند آن. اما تیم عملیات چگونه می‌تواند مطمئن شود که هر تیم واقعاً این کار را انجام داده است؟ و مسائلی مانند ارتقاهای یکپارچه چه می‌شود؟ اگر ابزار پایشِ استاندارد نیاز به ارتقا در سراسر سازمان داشته باشد، تیم‌ها چگونه باید آن را هماهنگ کنند؟

معماری `Hexagonal`

در الگوی معماری `Hexagonal` (شش‌ضلعی)، که در شکل ۲۰-۱ نشان داده شده، منطق حوزه در مرکز شش‌ضلعی قرار دارد و درگاه‌ها و مبدل‌هایی که به بخش‌های دیگر اکوسیستم متصل‌اند آن را احاطه کرده‌اند (در واقع، این الگو با نام دیگرِ الگوی **`Ports and Adapters`** (درگاه‌ها و مبدل‌ها) نیز شناخته می‌شود). بازنمایی بصریِ این الگو در شکل ۲۰-۱ آمده است.



شکل ۲۰-۱. الگوی معماری `Hexagonal`

خوانندگان تیزبین ممکن است متوجه شوند که تنها چهار ضلع از شش ضلع شش‌ضلعی به کار رفته است. پدیدآورندهٔ این الگو، Alistair Cockburn، در آغاز آن را به شکل شش‌ضلعی کشید و نامش را الگوی معماری `Hexagonal` گذاشت. او تقریباً بلافاصله پشیمان شد، چون نام `Ports and Adapters` گویاتر است؛ اما دیگر دیر شده بود. معماران بسیاری « `Hexagonal` » را نامی جذاب می‌دانستند و همین نام ماندگار شد.

اشتباه گرفتنِ الگو با پیاده‌سازی خطری رایج است و این نمونهٔ خوبی از آن است. معماری `Hexagonal`، وقتی برای توصیف میکروسرویس‌ها به کار رود، نقصی بالقوه جدی دارد؛ اما تنها برای کسانی که هدف اولیهٔ این الگو را می‌شناسند. معماری `Hexagonal` پیش از میکروسرویس‌های مدرن پدید آمد، اما شباهت‌های زیادی میان آن‌ها وجود دارد. یک تفاوت مهم نیز هست: وفاداری داده `(data fidelity)`. معماری `Hexagonal` با پایگاه‌داده صرفاً به‌عنوان مبدل دیگری رفتار می‌کند که می‌توان آن را متصل کرد. این الگو طرح‌وارهٔ داده را بخشی از منطق کسب‌وکار به شمار نمی‌آورد؛ به دلیل این برداشت نادرست (که در زمان پیدایش نام الگوی `Hexagonal` رایج بود) که پایگاه‌داده سازوکاری یکسره جداگانه است. Eric Evans با این بینش که طرح‌واره‌های پایگاه‌داده، صرف‌نظر از محل قرارگیری‌شان، باید برای بازتاب منطق کسب‌وکار سیستم تغییر کنند، این اشتباه را در کتابش **`Domain-Driven Design`** اصلاح کرد.

همین امر الگوی `Hexagonal` را به منبع دائمی سردرگمی میان معماران تبدیل می‌کند. هر بار که کسی از آن استفاده می‌کند، آیا جداسازیِ دغدغه‌های عملیاتی و حوزه‌ای را توصیف می‌کند، یا به خودِ الگو به معنای تحت‌اللفظی اشاره دارد که داده را جدا می‌کند و در نتیجه یکی از اصول بنیادین طراحی میکروسرویس‌ها را نقض می‌کند؟ استفاده از نام این

الگو به‌عنوان کوته‌نوشتی برای «جداسازیِ دغدغه‌های حوزه‌ای و عملیاتی» تا زمانی که در بافتار گمراه‌کننده نباشد اشکالی ندارد. با این حال، معماران امروز نیازی به این پیاده‌سازی ندارند. اکنون سازوکار مناسب‌تری برای پیاده‌سازیِ الگوی `Hexagonal` در اختیار داریم: الگوی مش سرویس `(Service Mesh)` .

مش سرویس

پیش‌تر در بخش «بازاستفادهٔ عملیاتی» در صفحهٔ ۳۳۴، رویکرد رایج معماری را برای جداسازیِ دغدغه‌های فنی از دغدغه‌های حوزه‌ای شرح دادیم: استفاده از الگوهای `Sidecar` و مش سرویس.

الگوی `Sidecar` فقط راهی برای جداکردنِ قابلیت‌های عملیاتی از حوزه‌ها نیست؛ بلکه یک الگوی بازاستفادهٔ متعامد است که به نوع خاصی از کوپلینگ می‌پردازد (نگاه کنید به «کوپلینگ متعامد» در صفحهٔ ۳۷۴). راه‌حل‌های معماری اغلب به چند نوع کوپلینگ نیاز دارند؛ مانند نمونهٔ کنونیِ ما، یعنی کوپلینگ حوزه‌ای در برابر کوپلینگ عملیاتی. یک الگوی **بازاستفادهٔ متعامد** `(Orthogonal Reuse)` راهی برای بازاستفاده از جنبه‌ای ارائه می‌کند که یک یا چند دغدغه در معماری آن را بازنمایی می‌کنند و در سازمان‌دهیِ سلسله‌مراتبیِ ترجیحی جا نمی‌گیرد. برای مثال، معماری‌های میکروسرویس حول حوزه‌ها سازمان می‌یابند، اما کوپلینگ عملیاتی مستلزم برش‌زدن از میان آن حوزه‌هاست. `sidecar` به معمار امکان می‌دهد این دغدغه‌ها را در لایه‌ای میان‌بُرشی اما یکدست در سراسر معماری جدا کند.

> **کوپلینگ متعامد `(Orthogonal Coupling)`**
>
> در ریاضیات، دو خط زمانی **متعامد** `(orthogonal)` هستند که با زاویهٔ قائمه یکدیگر را قطع کنند؛ و این به معنای استقلال آن‌ها نیز هست. در معماری نرم‌افزار، دو بخش از یک معماری ممکن است **به‌طور متعامد کوپل** باشند: ممکن است دو هدف متمایز داشته باشند که با این حال باید برای شکل‌دادن به یک راه‌حل کامل با هم تلاقی کنند. نمونهٔ آشکار در این فصل، دغدغه‌ای عملیاتی مانند پایش است که ضروری اما مستقل از رفتار حوزه، مانند پرداخت سبد خرید از کاتالوگ، است. شناختن کوپلینگ متعامد به معماران امکان می‌دهد نقاط تلاقی‌ای را بیابند که کمترین درهم‌تنیدگی را میان دغدغه‌ها پدید می‌آورند.

هرچند الگوی `Sidecar` انتزاع خوبی ارائه می‌کند، مانند همهٔ رویکردهای معماریِ دیگر مصالحه‌هایی دارد؛ چنان‌که در جدول ۲۰-۱ نشان داده شده است.

جدول ۲۰-۱. مصالحه‌های الگوهای `Sidecar` و مش سرویس

مزایا	معایب
راهی یکدست برای ایجاد کوپلینگِ جداشده فراهم می‌کند	باید برای هر پلتفرم یک `sidecar` پیاده‌سازی شود
هماهنگیِ یکدستِ زیرساخت را ممکن می‌کند	مؤلفهٔ `sidecar` ممکن است بزرگ یا پیچیده شود
مالکیت می‌تواند بر عهدهٔ هر تیم، متمرکز یا ترکیبی از این دو باشد	«رانش» پیاده‌سازی میان تیم‌های مستقل

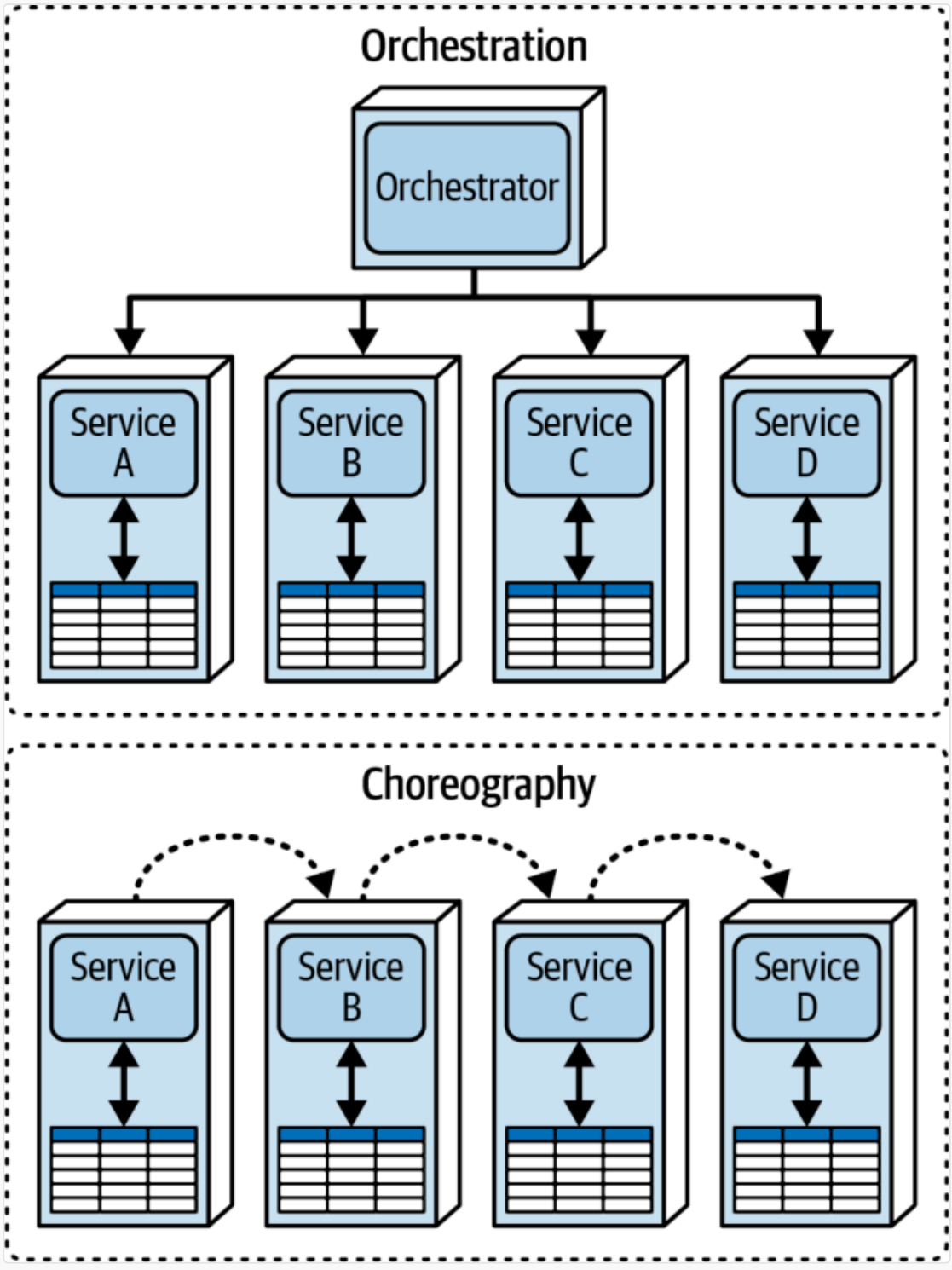
هر دو الگوی `Hexagonal` و مش سرویس شیوه‌هایی را برای پیاده‌سازیِ الگوی بازاستفاده با هدف جداکردنِ دغدغه‌های حوزه‌ای از عملیاتی نشان می‌دهند. پیاده‌سازیِ `Hexagonal` همه‌منظوره است، در حالی که مش سرویس به‌خوبی با میکروسرویس‌ها و دیگر معماری‌های توزیع‌شده سازگار است. نکتهٔ کلیدی برای معماران این است که نخست الگو، یعنی جداسازی، را شناسایی کنند و سپس تصمیم بگیرند که چگونه آن را به بهترین شکل در معماری خود پیاده‌سازی کنند.

ارتباطات

بسیاری از الگوهای معماری، از جمله الگوهای ارتباطی، از معماری رویدادمحور سرچشمه می‌گیرند و در هر معماری توزیع‌شده‌ای که از طریق پیام‌ها و/یا رویدادها ارتباط برقرار می‌کند، مانند آنچه در فصل‌های ۱۵ و ۱۸ بررسی شد، کاربرد دارند. در واقع، در هر دو فصل نمونه‌هایی از ارتباطات دیده‌اید؛ فقط آن‌ها را به‌عنوان الگوهایی مشخص معرفی نکرده‌ایم، زیرا معماران معمولاً الگوها را بی‌آنکه متوجه باشند پیاده‌سازی می‌کنند. هرچه باشد، الگوها راه‌حل‌هایی برای مسائل رایج‌اند.

ارکستراسیون در برابر کرئوگرافی

دو الگوی ارتباطی را در نظر بگیرید که پیش‌تر هم در بخش «معماری رویدادمحورِ میانجی‌دار» در صفحهٔ ۲۵۸ و هم در بخش «کرئوگرافی و ارکستراسیون» در صفحهٔ ۳۴۱ دیده‌اید: کرئوگرافی و ارکستراسیون، که در شکل ۲۰-۲ خلاصه شده‌اند.



شکل ۲۰-۲. ارکستراسیون و کرئوگرافی در یک معماری میکروسرویس

در هر یک از گردش‌کارهای هم‌ریختی که در شکل ۲۰-۲ نشان داده شده، چهار سرویس حوزه (سرویس‌های `A` تا `D`) باید برای شکل‌دادن به یک گردش‌کار با هم همکاری کنند. در حالت ارکستراسیون، سرویس جداگانه‌ای نیز وجود دارد که در نقش هماهنگ‌کنندهٔ گردش‌کار عمل می‌کند: هماهنگ‌کننده `(orchestrator)`.

هرچند این ارتباط را هم با عنوان ارکستراسیون و هم با عنوان میانجی‌گری (mediation) توصیف کرده‌ایم، الگو همان است. توانایی تشخیص الگوهایی که درون پیاده‌سازی‌ها پنهان‌اند به سود معماران است، زیرا مصالحه‌های آن‌ها را آشکارتر می‌کند.

توجه کنید که هنگام شرح مصالحه‌های میانجی‌گری و ارکستراسیون، به بسیاری از مسائل یکسان اشاره کردیم. مزایا را در اینجا خلاصه می‌کنیم:

- **گردش‌کار متمرکز (Centralized workflow):** با افزایش پیچیدگی، معماران از به‌کارگیریِ یک مؤلفهٔ یکپارچه برای حالت، رفتار و شرط‌های مرزی (boundary conditions) سود می‌برند.
- **مدیریت خطا (Error handling):** مدیریت خطا، که بخش مهمی از بسیاری از گردش‌کارهای حوزه است، با وجود یک مالکِ حالت برای گردش‌کار تسهیل می‌شود.
- **بازیابی‌پذیری (Recoverability):** از آنجا که هماهنگ‌کننده حالت گردش‌کار را پایش می‌کند، اگر یک یا چند سرویس حوزه دچار قطعیِ کوتاه‌مدت شوند، معمار می‌تواند منطقی برای تلاش دوباره (retry) بیفزاید.
- **مدیریت حالت (State management):** وجود هماهنگ‌کننده حالت گردش‌کار را قابل پرس‌وجو می‌کند و جایی برای گردش‌کارهای دیگر و حالت‌های گذرای دیگر فراهم می‌آورد.

معایب کلیِ ارکستراسیون عبارت‌اند از:

- **پاسخ‌گویی (Responsiveness):** همهٔ ارتباطات باید از هماهنگ‌کننده عبور کنند و این ممکن است گلوگاهی در توان عملیاتی (throughput) پدید آورد که به پاسخ‌گویی آسیب می‌زند.
- **تحمل خطا (Fault tolerance):** هرچند ارکستراسیون بازیابی‌پذیریِ سرویس‌های حوزه را بهبود می‌بخشد، یک نقطهٔ شکست واحد (single point of failure) بالقوه برای گردش‌کار پدید می‌آورد. می‌توان با افزونگی (redundancy) به این مشکل پرداخت، اما این کار نیز پیچیدگی بیشتری می‌افزاید.
- **مقیاس‌پذیری (Scalability):** این سبک ارتباطی به خوبیِ کرئوگرافی مقیاس‌پذیر نیست، زیرا هماهنگ‌کننده نقاط هماهنگیِ بیشتری می‌افزاید و همین موازی‌سازیِ (parallelism) بالقوه را کاهش می‌دهد.
- **کوپلینگ سرویس‌ها (Service coupling):** وجود یک هماهنگ‌کنندهٔ مرکزی کوپلینگ تنگ‌تری میان آن و مؤلفه‌های حوزه پدید می‌آورد؛ کوپلینگی که گاهی ضروری است، اما در معماری‌های میکروسرویس ناپسند شمرده می‌شود.

به همین ترتیب، کرئوگرافی را هم در میکروسرویس‌ها و هم در معماری‌های رویدادمحور بررسی کردیم. مصالحه‌های گردش‌کارهای کرئوگرافی‌شده عبارت‌اند از:

- **پاسخ‌گویی:** این سبک ارتباطی گلوگاه‌های منفرد کمتری دارد و از این رو فرصت‌های بیشتری برای موازی‌سازی فراهم می‌کند.
- **مقیاس‌پذیری:** نبودِ نقاط هماهنگی مانند هماهنگ‌کننده‌ها، مقیاس‌پذیریِ مستقل‌تری را ممکن می‌کند.

- **تحمل خطا:** نبودِ یک هماهنگ‌کنندهٔ واحد به معمار امکان می‌دهد برای بهبود تحمل خطا از چند نمونه استفاده کند. البته معمار می‌تواند چند هماهنگ‌کننده بسازد، اما از آنجا که همهٔ ارتباطات باید از آن‌ها عبور کنند، چند هماهنگ‌کننده نسبت به سطح کلیِ تحمل خطای گردش‌کار حساس‌ترند.
- **جداسازیِ سرویس‌ها `(Service decoupling)`:** نبودِ هماهنگ‌کننده یعنی کوپلینگ کمتر.

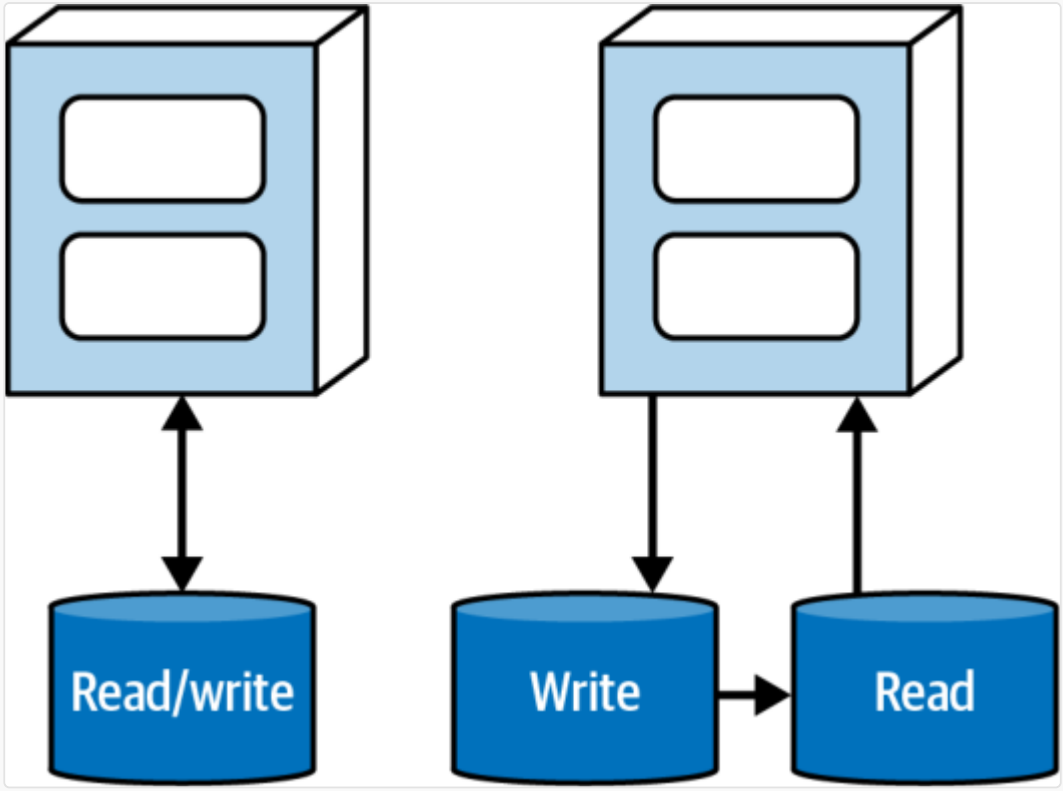
معایب سبک ارتباطیِ کرئوگرافی عبارت‌اند از:

- **گردش‌کار توزیع‌شده `(Distributed workflow)`:** نبودِ مالکی برای گردش‌کار، مدیریت خطاها و دیگر شرط‌های مرزی را دشوارتر می‌کند.
- **مدیریت حالت:** نبودِ نگه‌دارندهٔ متمرکزِ حالت، مدیریت مستمر حالت را دشوار می‌کند.
- **مدیریت خطا:** مدیریت خطا بدون هماهنگ‌کننده دشوارتر است، زیرا سرویس‌های حوزه باید دانش بیشتری دربارهٔ گردش‌کار داشته باشند.
- **بازیابی‌پذیری:** بدون هماهنگ‌کننده‌ای که تلاش‌های دوباره و دیگر اقدام‌های جبرانی را انجام دهد، بازیابی‌پذیری دشوارتر می‌شود.

این دو الگو تمایزی را که میان سبک‌ها و الگوها گذاشتیم به‌خوبی نشان می‌دهند. هر معماری توزیع‌شده‌ای می‌تواند از هر یک از این الگوهای ارتباطی استفاده کند و معماران باید بدانند چگونه مصالحه‌های آن‌ها را ارزیابی کنند. قانون دوم معماری نرم‌افزار `(Second Law of Software Architecture)` را به یاد بیاورید: تحلیل مصالحه را نمی‌توانید یک بار انجام دهید و تمام‌شده بدانید. این دو الگو همچنین نشان می‌دهند که الگوهای رایج همه‌جا وجود دارند؛ و دقیقاً به همین دلیل رایج‌اند.

CQRS

یکی دیگر از الگوهای ارتباطیِ رایج که در بسیاری از معماری‌های توزیع‌شده (و چند معماری تک‌پارچه) دیده می‌شود، **تفکیک مسئولیت فرمان و پرس‌وجو** (Command-Query-Responsibility-Segregation یا `CQRS`) است. این الگوی سادهٔ ارتباطی و داده‌ای، ارتباطی را که معمولاً به‌صورت یکپارچه با پایگاه‌داده برقرار می‌شود به دو بخش تقسیم می‌کند؛ چنان‌که در شکل ۲۰-۳ نشان داده شده است.



شکل ۲۰-۳. کلاینت/سرور در برابر `CQRS`

در شکل ۲۰-۳، ساختار سمت چپ تعامل دادهٔ رایجِ کلاینت/سرور `(client/server)` را نشان می‌دهد که در آن برنامهٔ کاربردی از پایگاه‌داده پرس‌وجو می‌کند و با استفاده از پایگاه‌داده به‌عنوان بخشی از زیرساخت برنامه، نوشتن‌های تراکنشی را انجام می‌دهد. هرچند این الگویی رایج است، برخی سیستم‌ها تفاوت‌های چشمگیری میان حجم خواندن و نوشتن دارند، یا می‌خواهند به دلایل امنیتی و دغدغه‌های دیگر، خواندن‌ها را از نوشتن‌ها جدا کنند. برای چنین سیستم‌هایی، `CQRS`، که در سمت راست شکل ۲۰-۳ نشان داده شده، این مسئله را حل می‌کند.

`CQRS` نوشتن‌ها را در یک مخزن داده (معمولاً یک پایگاه‌داده؛ گاهی زیرساختی دیگر، مانند یک صف پیام بادوام) جدا می‌کند که داده‌ها را (معمولاً به‌صورت ناهم‌گام) با پایگاه‌دادهٔ دیگری هم‌گام می‌سازد و آن پایگاه‌داده به درخواست‌های خواندن پاسخ می‌دهد.

معماران با جداکردنِ خواندن‌ها و نوشتن‌ها می‌توانند، بسته به داده، ویژگی‌های معماریِ متفاوتی را از هم جدا کنند. این کار همچنین به آن‌ها امکان می‌دهد در صورت نیاز برای هر پایگاه‌داده از مدل‌های دادهٔ `(data models)` متفاوتی استفاده کنند.

`CQRS` نمونهٔ خوبی از یک الگوی ارتباط داده است که امکان ویژگی‌های معماریِ متفاوت را برای انواع مختلف قابلیت‌های داده، دغدغه‌های امنیتی یا عوامل دیگری که از جداسازیِ فیزیکی سود می‌برند فراهم می‌کند.

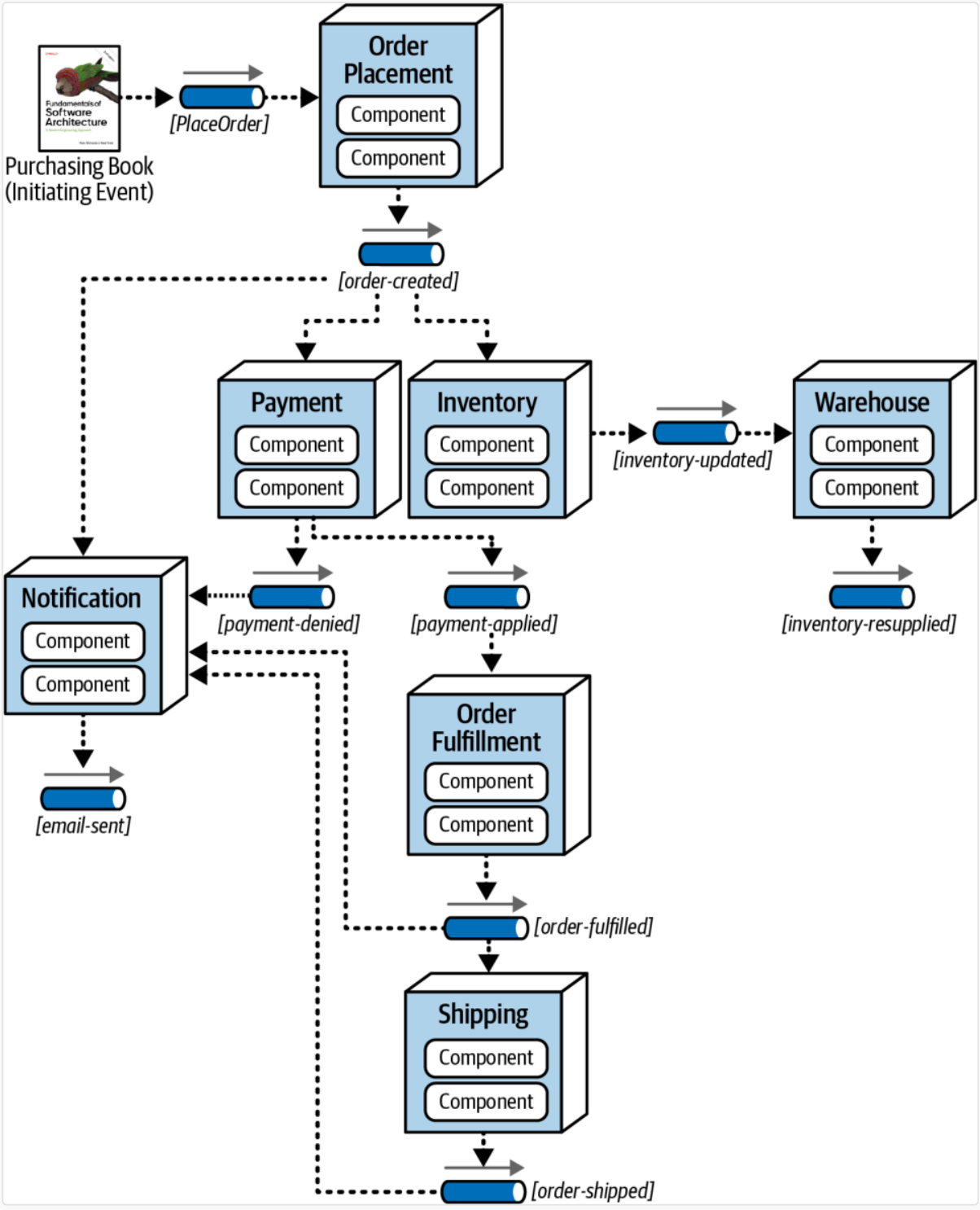
زیرساخت

معماری نرم‌افزار در هر جایی که تیم‌ها راه‌حل‌های سودمند و وابسته به زمینه‌ای برای مسائل رایج یافته‌اند الگو دارد و این الگوها اغلب با بخش‌های دیگر اکوسیستم تلاقی می‌کنند (برای بحثی کامل، فصل ۲۶ را ببینید).

معماران به کوپلینگ اهمیت می‌دهند: میان مؤلفه‌ها، عناصر داده، `API`ها... و زیرساخت؛ چنان‌که الگوی کارگزار حوزه‌ای `(Broker-Domain)` نمونه‌ای از آن است.

الگوی کارگزار حوزه‌ای

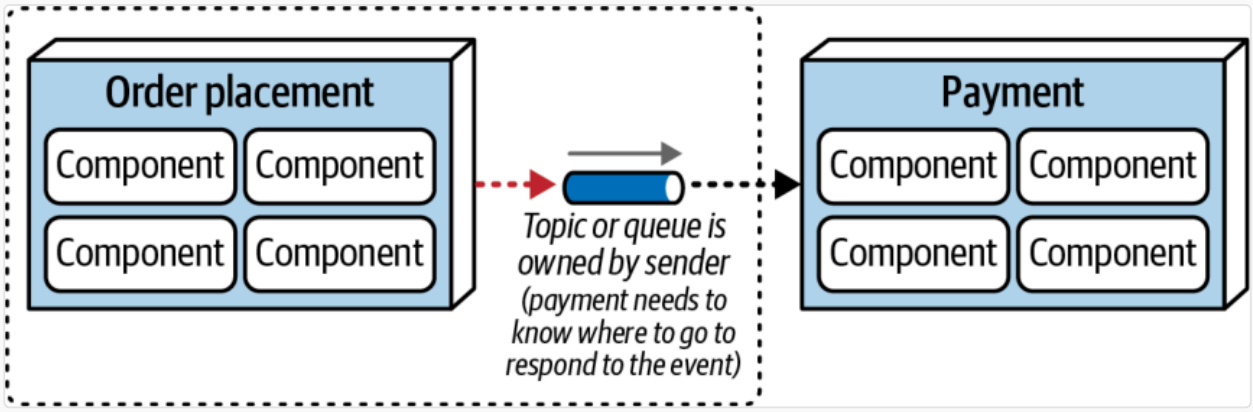
در این بخش، همان گردش‌کار ثبت سفارش را بررسی می‌کنیم، این بار با پیاده‌سازی در یک معماری رویدادمحور؛ چنان‌که در شکل ۲۰-۴ نشان داده شده است.



شکل ۲۰-۴. گردش‌کار ثبت سفارش که در `EDA` پیاده‌سازی شده است

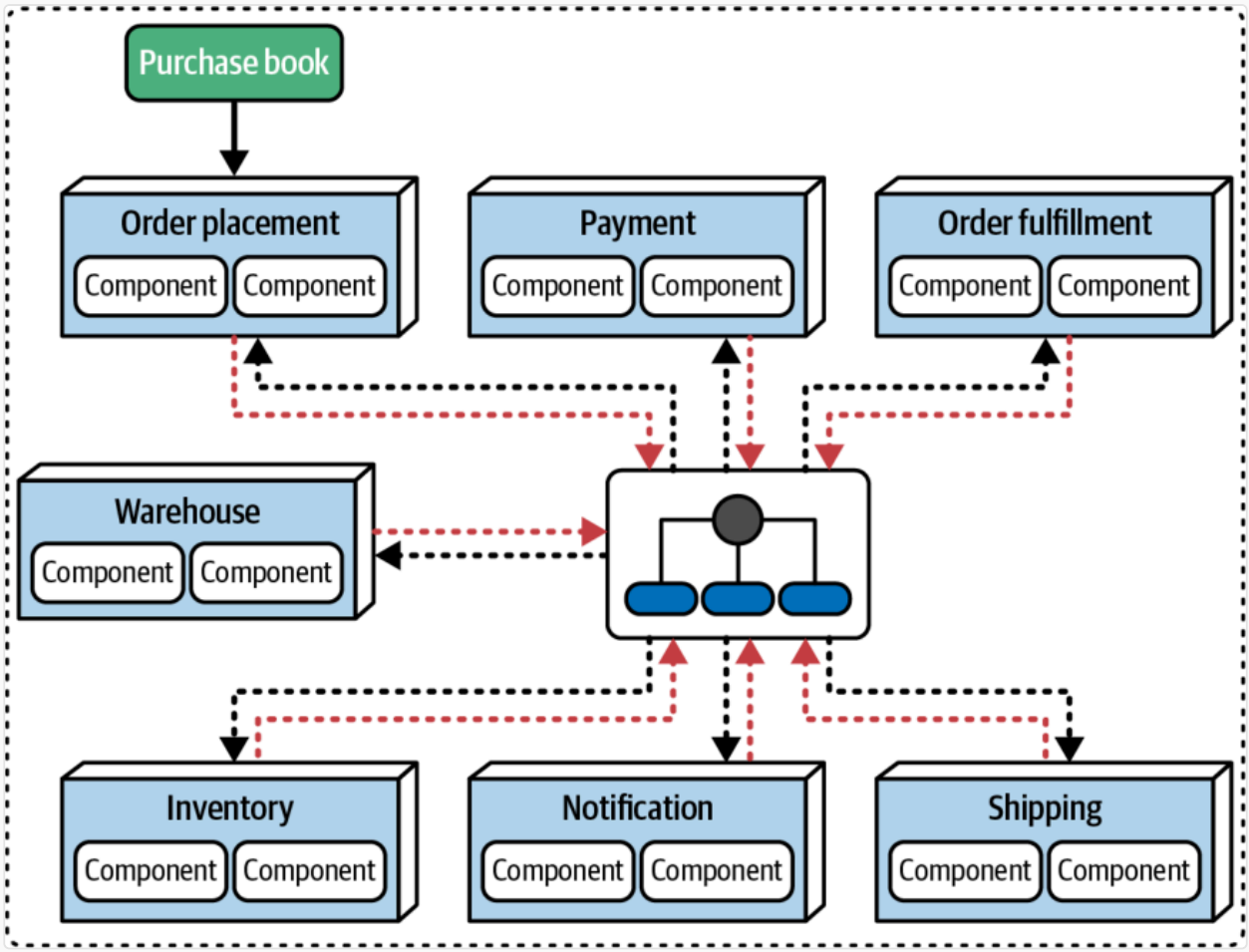
همان‌طور که دیده‌اید، `EDA` برای ارتباط میان سرویس‌ها از رویدادها استفاده می‌کند؛ بنابراین گرداننده‌های رویداد `(event handlers)` برای ساختنِ گردش‌کار باید مشترک سرویس‌های مناسب شوند. گرداننده‌های رویداد با کارگزارها `(brokers)` پیاده‌سازی می‌شوند که بخشی از زیرساخت معماری‌اند. در `EDA`، موضوع `(topic)` یا صف `(queue)` معمولاً متعلق به فرستنده است: برای مثال، `Payment` باید نشانی یک موضوع را بداند تا مشترک آن شود.

در شکل ۲۰-۵، `OrderPlacement` «مالک» کارگزاری است که پردازنده‌های دیگر مشترک آن می‌شوند؛ به بیان دیگر، زیرساخت لازم برای پشتیبانی از این سرویس شامل کارگزار است. اگر سیستمی برای همهٔ ارتباطات تنها از یک کارگزار استفاده کند، همهٔ سرویس‌هایش به یک بخش واحد از زیرساخت وابسته‌اند.



شکل ۲۰-۵. در کرئوگرافیِ رویدادمحور، موضوع یا صف معمولاً متعلق به فرستنده است

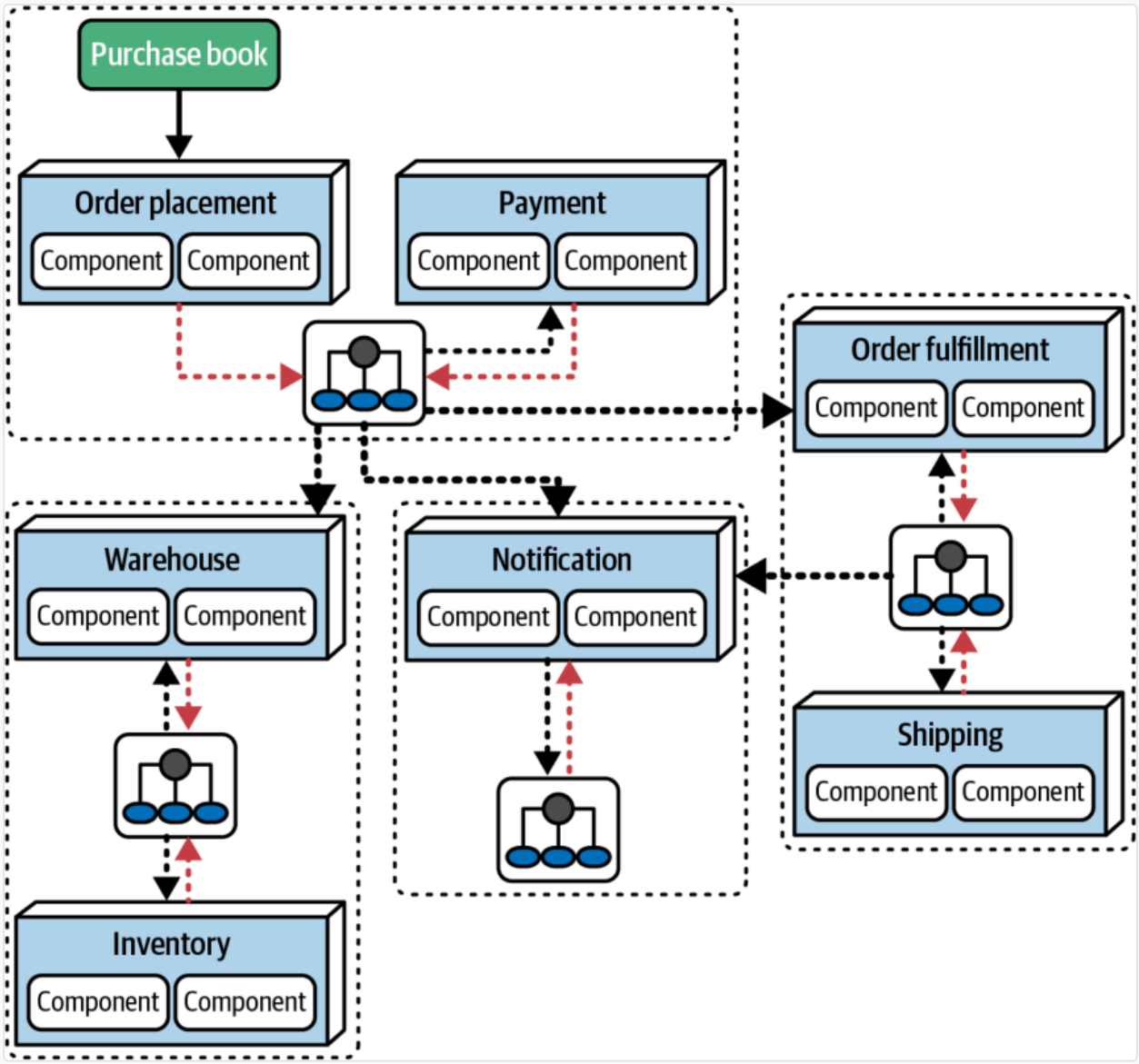
زیرساخت گردش‌کاری که در شکل ۲۰-۶ نشان داده شده یک کارگزار واحد دارد؛ یعنی هر پردازندهٔ رویداد «می‌داند» برای مشترک‌شدن در همکاران گردش‌کار به کجا برود. یک کارگزار واحد همچنین جایگاهی واحد برای ثبت وقایع، پایش و دیگر سازوکارهای حاکمیتی فراهم می‌کند.



شکل ۲۰-۶. استفاده از یک کارگزار برای کل گردش‌کار

با این حال، یکی از اهداف معماری توزیع‌شده بهبود تحمل خطاست. اگر کارگزار واحد در شکل ۲۰-۶ از کار بیفتد چه می‌شود؟ کل گردش‌کار متوقف می‌شود. مشکل بالقوهٔ دیگر، مقیاس است. اگر همهٔ پیام‌ها باید از یک کارگزار واحد عبور کنند، این خطر وجود دارد که با افزایش حجم پیام‌ها، کارگزار زیر بار غرق شود.

رویکرد جایگزین، الگوی **کارگزار حوزه‌ای** `(Domain-Broker pattern)` است که با زیرساخت به شیوه‌ای مشابه دانه‌بندی حوزه‌ها رفتار می‌کند. شکل ۲۰-۷ را در نظر بگیرید.



شکل ۲۰-۷. بهره‌گیری از الگوی کارگزار حوزه‌ای برای واگذاریِ مالکیت زیرساخت

در معماری جایگزینی که در شکل ۲۰-۷ نشان داده شده، هر گروه از سرویس‌های مرتبط از یک کارگزار مشترک استفاده می‌کنند و این بازتاب افراز حوزه‌ایِ کلیِ معماری است. این راه‌حل همچنان امکان کشف `(discovery)` خوب را فراهم می‌کند و در عین حال تحمل خطا، مقیاس‌پذیری، کشسانی و شمار زیادی از دیگر ویژگی‌های عملیاتی معماری را افزایش می‌دهد. اما توجه کنید که هیچ‌یک از این دو رویکرد «بهترین شیوه» نیست. مصالحه‌های الگوی **کارگزار واحد** `(Single-Broker pattern)` در جدول ۲۰-۲ آمده است.

جدول ۲۰-۲. مصالحه‌های الگوی کارگزار واحد

مزایا	معایب
کشف متمرکز	تحمل خطا
کمترین زیرساختِ ممکن	محدودیت‌های توان عملیاتی

الگوی کارگزار حوزه‌ای نیز مصالحه‌هایی دارد که در جدول ۲۰-۳ نشان داده شده‌اند.

جدول ۲۰-۳. مصالحه‌های الگوی کارگزار حوزه‌ای

مزایا	معایب
جداسازیِ بهتر	کشف دشوارترِ صف‌ها/موضوع‌ها
انطباق با مرزهای حوزه	زیرساخت بیشتر = هزینهٔ بیشتر
مقیاس‌پذیری بیشتر	اجزای متحرکِ بیشتر برای نگهداری

معماران هنگام تصمیم‌گیری دربارهٔ اینکه کدام‌یک از این الگوهای زیرساختی برای سیستم مشخصشان معقول‌تر است، باید میان کشف‌پذیری و نیاز به جداسازیِ حوزه‌ها توازن برقرار کنند.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
`Architecture Style`	سبک معماری
`Architecture Pattern`	الگوی معماری
`Best Practice`	بهترین شیوه
`Domain Coupling`	کوپلینگ حوزه‌ای
`Operational Coupling`	کوپلینگ عملیاتی
`Loose Coupling`	کوپلینگ سست
`Name-Value Pair`	جفت نام-مقدار

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Semantic Drift	رانش معنایی
Hexagonal Architecture (pattern)	معماری Hexagonal (شش‌ضلعی)
Ports and Adapters (pattern)	Ports and Adapters (درگاه‌ها و مبدل‌ها)
Data Fidelity	وفاداری داده
Service Mesh	مش سرویس
Orthogonal Reuse (pattern)	بازاستفادهٔ متعامد
Orthogonal Coupling	کوپلینگ متعامد
Orchestrator	هماهنگ‌کننده
Mediation	میانجی‌گری
Centralized Workflow	گردش‌کار متمرکز
Boundary Condition	شرط مرزی
Error Handling	مدیریت خطا
Recoverability	بازیابی‌پذیری
Retry	تلاش دوباره
State Management	مدیریت حالت
Responsiveness	پاسخ‌گویی
Throughput	توان عملیاتی
Fault Tolerance	تحمل خطا
Single Point of Failure	نقطهٔ شکست واحد
Redundancy	افزونگی
Scalability	مقیاس‌پذیری
Parallelism	موازی‌سازی
Service Coupling	کوپلینگ سرویس‌ها

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Service Decoupling	جداسازیِ سرویس‌ها
Distributed Workflow	گردش‌کار توزیع‌شده
Second Law of Software Architecture	قانون دوم معماری نرم‌افزار
Command-Query-Responsibility-Segregation (CQRS)	تفکیک مسئولیت فرمان و پرس‌وجو (CQRS)
Client/Server	کلاینت/سرور
Data Model	مدل داده
Broker-Domain / Domain-Broker (pattern)	الگوی کارگزار حوزه‌ای
Event Handler	گردانندهٔ رویداد
Broker	کارگزار
Topic	موضوع
Queue	صف
Discovery (tool)	کشف
Single-Broker (pattern)	الگوی کارگزار واحد